

Лабораторная работа №3 РАБОТА С ПОТОКАМИ

Цель работы: научиться создавать и уничтожать вычислительные потоки, а также управлять ими.

Понятие *потока* несколько различается в системах Linux и Windows. Более того, диспетчеризация потоков может быть реализована как на уровне пользователя (облегченные потоки Windows, *fibers*), так и на уровне ядра. В дальнейшем будем рассматривать исключительно потоки на уровне ядра.

В Windows процессы на самом деле не выполняют никакого кода, а являются лишь контейнерами для вычислительных потоков. Потоки представляют собой отдельные объекты ядра, коренным образом отличающиеся от процессов. Любой процесс может иметь несколько параллельно выполняющихся внутри него потоков, объединенных общим адресным пространством, общими таблицами дескрипторов процесса и т.д. Процесс должен содержать как минимум один поток, иначе он завершается. При создании процесса создается первичный поток, который вызывает входную функцию *main()*, *WinMain()* и т.д. При завершении входной функции управление вновь передается стартовому коду, и тот вызывает функцию *Exit-Process()*, завершая текущий процесс.

Работа с потоками в Windows, несмотря на основополагающие архитектурные отличия, во многом аналогична работе в Linux. В потоках Windows тоже используется понятие потоковой функции. Поток создается следующей функцией WinAPI:

```
HANDLE CreateThread(PSECURITY_ATTRIBUTES lpThreadAttributes,  
DWORD dwStackSize, PTHREAD_START_ROUTINE lpStartAddress, PVOID  
lpParameter, DWORD dwCreationFlags, PDWORD lpThreadId);
```

Функция принимает следующие параметры:

- *lpThreadAttributes* – атрибуты безопасности потока;
- *dwStackSize* – размер стека, выделяемого под поток;
- *lpStartAddress* – адрес потоковой функции, созданной в соответствии с прототипом *DWORD WINAPI ThreadFunc(PVOID pvParam)*;
- *lpParameter* – указатель на параметры, передаваемые функции потока при его создании;
- *dwCreationFlags* – флаг создания потока (поток запускается немедленно или создается в остановленном состоянии);
- *lpThreadId* – указатель, по которому будет записан идентификатор созданного потока.

Завершается поток одним из следующих способов:

- функция потока возвращает управление;
- поток вызывает функцию *ExitThread()*;

- текущий (или другой) поток вызывает функцию *TerminateThread()*;
- процесс, в котором выполняется поток, завершается.

Важно отметить, что создание потока при помощи функции *CreateThread()* может привести к некорректной работе приложения, если одновременно используются глобальные функции и переменные стандартной библиотеки C/C++ (*errno*, *strtok*, *strerror* и т.д.). В таких случаях рекомендуется прибегнуть к функциям *_beginthreadex* и *_endthreadex*, которые реализуют аналогичную функциональность, но безопасны в работе.

Для решения проблемы синхронизации в Windows также применяются мьютексы, которые здесь, по сути, представляют собой бинарные семафоры и предназначены в первую очередь для взаимодействия процессов. Тем не менее использование подобных глобальных объектов в многопоточном приложении не всегда оправдано.²

Более подходящим механизмом для синхронизации потоков являются критические секции,² логика работы которых во многом сходна с работой мьютексов в pthreads. Для работы с критическими секциями существует пять основных функций:

- *InitializeCriticalSection* – создание и инициализация объекта «критическая секция»;
- *EnterCriticalSection* – вход в критическую секцию. Это действие аналогично блокировке мьютекса. Если блокируемая критическая секция уже используется каким-либо потоком, функция ждет ее освобождения;
- *LeaveCriticalSection* – выход из критической секции;
- *TryEnterCriticalSection* – попытка входа в критическую секцию. Функция аналогична *EnterCriticalSection()*. Если критическая секция занята, возвращается 0, в противном случае – ненулевое значение;
- *DeleteCriticalSection* – удаление критической секции.

ЗАДАНИЕ

Задание аналогично заданию из лабораторной работы №2, но с реализацией с помощью потоков.